



SIKSHA 'O' ANUSANDHAN

(Deemed To Be University)

Accredited (3rd Cycle) by NAAC with A++ Grade

Admission Batch: **2023**

Session: **2025–26**

Major Project Report

of

MACHINE LEARNING CONCEPT -2 (CSE 3968)

On

**Disease Prediction System
Using Machine Learning**

Submitted by:

- | | |
|-----------------------|------------|
| 1) SASWAT BARAI | 2341002028 |
| 2) DEBANANDA BEHERA | 2341002103 |
| 3) AYAN BHATTACHARJEE | 2341002172 |
| 4) AMIT KUMAR BHUKTA | 2341002180 |

Section: **23412G2**

Semester: **6TH**

Branch: **CSE**

Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER), Siksha 'O' Anusandhan
Deemed to be University
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha – 751030

(May, 2026)

DECLARATION

We hereby declare that the project report entitled

“Disease Prediction System Using Machine Learning”

is a bonafide record of the work carried out by us during the 6th semester of the Bachelor of Technology program in Computer Science and Engineering for the course **Machine Learning Concept - 2 (CSE 3968)**, under the regulations of Siksha ‘O’ Anusandhan Deemed to be University.

This work has been carried out under the guidance of the project supervisor and has not been submitted elsewhere for any academic evaluation or certification. We further declare that there are **no competing interests** associated with this project.

Submitted by:

SASWAT BARAI (Regd. No: 2341002028) Signature: _____

DEBANANDA BEHERA (Regd. No: 2341002103) Signature: _____

AYAN BHATTACHARJEE (Regd. No: 2341002172) Signature: _____

AMIT KUMAR BHUKTA (Regd. No: 2341002180) Signature: _____

Project Supervisor:

Name: Kumari Manju

Signature: _____

Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER)
Siksha ‘O’ Anusandhan Deemed to be University
Bhubaneswar, Odisha

Date: _____

Contents

1	Abstract	5
2	Introduction	5
3	Problem Statement	6
4	Objectives	6
5	Literature Review	7
6	Dataset Description	8
6.1	Source	8
6.2	Dataset Characteristics	8
6.3	Feature Representation	8
7	Methodology	9
7.1	Data Collection	9
7.2	Data Preprocessing	9
7.3	Exploratory Data Analysis	10
7.4	Model Development	11
7.4.1	Decision Tree Classifier	11
7.4.2	Random Forest Classifier	12
7.4.3	XGBoost Classifier	12
7.4.4	Stacking Ensemble Classifier	13
7.5	Model Evaluation	13
8	System Architecture	15
9	Implementation	16
9.1	Tools and Technologies	16
9.2	Implementation Steps	16
9.3	Key Code Snippet	17
10	Results	18
10.1	Model Accuracy Comparison	18
10.2	Confusion Matrices	19
10.3	Feature Importance	19
11	Discussion	21
11.1	What the Numbers Actually Mean	21
11.2	What Works Well and What Does Not	22
12	Applications	23
13	Future Scope	24
14	Conclusion	25
15	Roles and Contributions	26

16 References	26
17 Appendix — Complete Source Code	28
17.1 Sample Dataset Records	28
17.2 Cell 1 — Imports	28
17.3 Cell 2 — Data Loading and Feature Matrix Construction	28
17.4 Cell 3 — Missing Value Check	29
17.5 Cell 4 — Symptom Dropout and Age Feature Generation	29
17.6 Cell 5 — Label Encoding	31
17.7 Cell 6 — Train-Test Split	31
17.8 Cell 7 — Disease Class Distribution (EDA)	31
17.9 Cell 8 — Symptom Frequency and Age Distribution (EDA)	32
17.10Cell 9 — Correlation Heatmap (EDA)	33
17.11Cell 10 — Feature Engineering (Variance Filtering)	33
17.12Cell 11 — Decision Tree Training and Evaluation	33
17.13Cell 12 — Random Forest Training and Evaluation	34
17.14Cell 13 — XGBoost and Stacking Ensemble Training	34
17.15Cell 14 — Confusion Matrix Helper Function	35
17.16Cell 15 — Decision Tree Confusion Matrix	36
17.17Cell 16 — Random Forest Confusion Matrix	36
17.18Cell 17 — XGBoost and Stacking Ensemble Confusion Matrices	36
17.19Cell 18 — Model Accuracy Comparison Table	36
17.20Cell 19 — Model Accuracy Bar Chart	37
17.21Cell 20 — Feature Importance Plot	37
17.22Cell 21 — Final Results Summary	38

1 Abstract

Getting a proper diagnosis at the right time can genuinely be the difference between a smooth recovery and something far more serious. The frustrating reality, especially in smaller towns and rural pockets of India, is that most people simply do not have dependable access to a doctor each time they fall ill. This project was put together with that problem squarely in mind. We developed a **Disease Prediction System** that accepts a patient’s age and self-reported symptoms — represented as severity scores — and identifies the most probable disease from a pool of 41 conditions.

Our dataset, sourced from Kaggle [7], holds 4,925 patient records. Rather than encoding each symptom as a plain yes-or-no flag, we assigned severity weights ranging from 1 to 7 — giving the model a far richer, more realistic view of how unwell the patient actually is. Two design choices distinguish our pipeline from similar work: (1) an **age-aware feature** drawn from epidemiological age distributions for each disease, embedding genuine clinical knowledge directly into the input; and (2) a **Stacking Ensemble** [19] that uses a Decision Tree, Random Forest (200 estimators), and XGBoost as base learners, whose out-of-fold probability predictions are fed — alongside the original features (`passthrough=True`) — into a Logistic Regression meta-learner for the final decision.

Four classifiers were trained and evaluated on 985 held-out test samples. The Stacking Ensemble topped the board at **93.50%** accuracy (macro precision 0.94, macro recall 0.93), followed by Random Forest at **93.30%**, XGBoost at **91.47%**, and the Decision Tree at **85.58%**.

Keywords: Disease Prediction, Machine Learning, Stacking Ensemble, Random Forest, XGBoost, Decision Tree, Symptom Severity, Age-Aware Features, Multi-class Classification.

2 Introduction

We have all done it at some point — typed our symptoms into a search engine and come back with a list running from a mild stomach bug to something outright alarming. The core trouble is not that information is scarce; it is that raw symptom data is genuinely difficult to make sense of without either clinical training or a well-calibrated model doing the heavy lifting. Machine learning sits neatly in that middle ground, and that is what drew us toward this problem.

Over the past decade, researchers across the globe have been applying ML to clinical decision support [9, 12], and the motivation is hard to argue with. The World Health Organization estimates that over half of the world’s population lacks access to essential health services [18], and hospitals generate staggering volumes of patient data every single day — patterns buried within that data can surface diagnoses more quickly and consistently than any individual physician operating under time pressure. The snag is that most of this work never leaves the pages of academic journals. We wanted to honestly test what a clean, end-to-end classification pipeline could do on a publicly available symptom-disease dataset — no deep learning stacks, no cloud APIs, just robust classical machine learning applied thoughtfully.

We selected four algorithms that step up in complexity in a natural way, finishing with a meta-learning ensemble:

- **Decision Tree** [14] — easy to follow, straightforward to explain to a non-technical person, and a sensible baseline to measure everything else against.
- **Random Forest** [1] — builds a whole forest of 200 individual trees, each trained on a slightly different slice of the data, and combines their outputs to dampen individual errors.
- **XGBoost** [3] — goes a step further by building trees one after another, each focused on correcting whatever the current ensemble got wrong.
- **Stacking Ensemble** [19] — blends all three base models through a Logistic Regression meta-learner trained on out-of-fold probability outputs, with `passthrough=True` so the meta-learner can directly inspect the raw feature values as well.

Two additional elements were woven into the pipeline: (1) an **age-aware feature** sampled from clinically grounded age distributions for each disease, and (2) a **50% symptom dropout** step that mimics the incomplete symptom reporting seen in real-world clinical encounters. The system takes a vector of severity scores and patient age as input, and returns one predicted disease label from 41 possible conditions.

3 Problem Statement

Consider a patient who walks into a clinic saying they have had a fever for three days, their joints are aching, and they feel completely drained. That exact combination of symptoms could reasonably indicate malaria, dengue, typhoid, or any number of other conditions. Telling them apart without laboratory investigations is not straightforward, even for an experienced clinician.

The central question we are addressing: Given a patient’s self-reported symptom severity scores, can a machine learning model reliably predict which of 41 diseases they are most likely dealing with?

- **Input:** A 137-dimensional feature vector — 136 symptom severity scores (0 = symptom absent, 1–7 = clinical severity weight) plus one **age** value sampled from the epidemiological age range of each disease.
- **Output:** One of 41 disease labels (e.g., Malaria, Tuberculosis, Diabetes, and so on).
- **Task type:** Supervised multi-class classification.

What makes this genuinely difficult is not the modelling itself — it is the heavy symptom overlap between many disease pairs. The four hepatitis variants (B, C, D, and E) are a good example: from a symptom perspective alone, they look nearly identical to one another. This is precisely where severity-weighted encoding earns its place, since a patient with “fatigue” at severity 2 and another with the same symptom at severity 7 are presenting very differently, and that difference matters diagnostically.

4 Objectives

Before writing a single line of code, we sat down and agreed on what we actually wanted to accomplish:

1. Thoroughly clean and organise the raw symptom dataset into a form that machine learning classifiers can actually work with.

2. Train four different models on the exact same data split and preprocessing pipeline, so any performance differences are down to the algorithms and not the setup.
3. Evaluate each model properly — going beyond plain accuracy to include precision, recall, F1-score, and confusion matrices that show exactly where predictions go wrong.
4. Identify which model would genuinely be worth recommending for deployment, weighing predictive performance against how explainable the results are.
5. Understand *which symptoms contribute most* to the predictions by analysing feature importances extracted from the Random Forest.

5 Literature Review

The push to automate medical diagnosis actually predates modern machine learning by several decades. Rule-based expert systems in the 1980s tried to capture a physician’s reasoning by encoding clinical logic as explicit if-then statements. The fundamental weakness of those systems was rigidity — the moment a patient walked in with an unusual symptom combination not anticipated by the rulebook, performance fell apart. Data-driven machine learning addressed this head-on by learning directly from patient records rather than relying on hand-crafted rules.

Early symptom-based ML efforts continued to face challenges due to the inherently messy nature of clinical data. Kononenko [9] evaluated several classifiers on medical datasets and reported that even Naive Bayes — which assumes full statistical independence between features, a clearly incorrect assumption for symptoms — managed competitive accuracy. That finding is actually quite telling: if a model built on a fundamentally wrong assumption can perform reasonably well, a well-tuned tree ensemble working with severity-weighted features should do considerably better. During the same period, decision trees [14] became especially popular in clinical contexts because they produced human-readable rules. A clinician could look at the tree, spot a branch that seemed medically nonsensical, and raise it with the data scientist — an oversight loop that no black-box method supports.

The arrival of Random Forests [1] was a significant moment for structured classification problems. Breiman’s central insight was deceptively simple: the weaknesses of individual decision trees — overfitting, sensitivity to noise and outliers — largely cancel out when you combine many trees, each trained on a different random subsample of the data with a different random subset of features at each split. In a medical setting, where symptom severity values are inherently noisy and many symptoms co-occur across unrelated diseases, this variance-reduction property is especially valuable. Later work by Rajpurkar et al. [15] on radiology reinforced the finding that ensemble methods generalise far more reliably than standalone models when class distributions overlap. Liaw and Wiener [10] further demonstrated Random Forest’s robustness in high-dimensional feature spaces, which directly parallels our 135-feature symptom matrix.

Gradient boosting [5] followed a different path to similarly strong results. Where Random Forest constructs trees independently and averages them, XGBoost [3] builds trees in strict sequence, with every successive tree trained to minimise the residual loss of the current ensemble. For a 41-class problem in which several hepatitis variants look nearly identical from a symptom standpoint, this iterative error-correction mechanism is a natural match — the model is given multiple passes to refine its output on the cases it consistently gets

wrong. The stacking generalisation principle underlying our meta-learner was formalised by Wolpert [19] and later extended by Ting and Witten [16], who demonstrated that combining diverse base learners through a trained combiner consistently outperforms individual models on structured classification tasks.

The gap in existing literature is mostly one of ambition and scope. A substantial portion of published clinical ML systems tackle only one or two diseases, or simplify the problem to a binary healthy-versus-sick question. Our system handles 41 disease classes in a single unified model, which is both harder and more practically useful. The severity-weighted feature encoding we adopted also goes well beyond the simple binary presence/absence flags that dominate prior work, giving the classifier richer signal to work with.

6 Dataset Description

6.1 Source

For this project we used the publicly available dataset published on Kaggle by *itachi9604* [7], accessed via the GitHub mirror maintained by *amMistic* [8]. The data is split across two files:

- `disease_raw.csv` — 4,925 patient records, each one listing up to 17 symptoms by name.
- `symptom_severity.csv` — a lookup table assigning each of the 133 known symptoms a severity weight on a scale of 1 to 7.

6.2 Dataset Characteristics

Table 1: Summary of the Disease Symptom Prediction Dataset [7]. The dataset is sourced from two CSV files: `disease_raw.csv` (patient symptom records) and `symptom_severity.csv` (clinical severity weights 1–7 per symptom). After severity-weighted vectorisation, 50% symptom dropout, age-feature augmentation, and zero-variance filtering, the final model-ready matrix contains **135 features** across **4,925 records** spanning **41 disease classes**. The 80/20 stratified split (Section 7.2) ensures all 41 classes are proportionally represented in both training and evaluation subsets.

Property	Details
Total Records	4,925
Disease Classes	41
Symptom Features	136 (after vectorisation) + 1 age feature = 137 total
Feature Type	Severity-weighted (0 = absent, 1–7 = present) + patient age
Features after Filtering	135 (zero-variance features removed via <code>VarianceThreshold</code>)
Missing Values	0 (after preprocessing)
Train / Test Split	80% / 20% stratified (3,940 train 985 test)

6.3 Feature Representation

The raw CSV stores each patient’s condition as up to 17 symptom names spread across individual columns (`Symptom_1` through `Symptom_17`). No classifier can consume text

strings arranged that way — we needed to convert everything into a fixed-length numerical vector first.

Our approach was straightforward: collect every unique symptom name appearing anywhere across all records (136 in total after vectorisation), then construct a 136-column matrix where each column represents one symptom. For each patient, we look up their reported symptoms in the severity table and fill the relevant columns with the corresponding weight. Symptoms not reported by that patient stay at 0.

The decision to use severity weights rather than plain binary flags was a deliberate one. Take two patients who both report fever: one has a severity score of 2 (mild, likely low-grade), the other scores 7 (dangerously high). Diagnostically these are quite different situations — the second patient is far more likely to be dealing with a serious infection. A binary 0/1 encoding collapses both into the same value and throws that distinction away entirely. Severity encoding preserves it.

On top of this, we introduced a **patient age feature** anchored in clinical epidemiology. Each of the 41 diseases was assigned the age range in which it most commonly presents (e.g. Chicken pox: 2–12, Heart attack: 45–80, Acne: 13–25). A realistic patient age was then sampled from that range for every record, giving the model some demographic context to work with — which is particularly helpful for distinguishing diseases that share many overlapping symptoms but affect different age groups. The total feature count after this addition is **137** (136 symptom severity scores plus the age column), reduced to **135** once two zero-variance features are removed.

7 Methodology

7.1 Data Collection

Both dataset files were pulled directly from their publicly accessible GitHub URLs using `curl`. No API tokens, login credentials, or institutional access were required. The files were stored locally inside a `dataset/` subfolder within the project directory, keeping paths consistent across all notebook cells.

7.2 Data Preprocessing

Going from raw CSV files to a clean, model-ready matrix involved several careful steps:

1. **Severity-Weighted Feature Construction:** We walked through every symptom column for every patient record, looked up the clinical severity weight for each reported symptom, and filled the corresponding position in a 136-column feature matrix. Symptoms that a patient did not report were left as 0.
2. **Incomplete Symptom Simulation:** In practice, patients rarely disclose every symptom they are experiencing — some are forgotten, some are considered trivial, and some are embarrassing to mention. To reflect this, we randomly zeroed out 50% of the non-zero entries in the feature matrix (fixed seed of 42 for full reproducibility). This step makes the classification task harder and considerably more realistic.
3. **Age Feature Generation:** A patient age column was added by sampling from the disease-specific epidemiological age range for each record (e.g. Chicken pox: 2–12, Heart attack: 45–80). This encodes clinical prior knowledge directly into the input,

helping the model separate diseases that look identical in terms of symptoms but typically affect very different age groups.

4. **Label Encoding:** The target column (`prognosis`) stores disease names as free text. We applied `LabelEncoder` to convert these into integer class indices — a prerequisite for all four classifiers in our pipeline.
5. **Train-Test Split:** The data was split 80/20 with stratification, ensuring every disease class appears in the correct proportion in both subsets. This produced 3,940 training samples and 985 test samples.
6. **Variance Thresholding:** After the dropout simulation, two symptom features turned out to have zero variance across the entire training set — meaning they carried no information. We removed them using `VarianceThreshold`, reducing the final feature count to **135** (134 symptom features plus the age feature).

7.3 Exploratory Data Analysis

Before jumping straight into modelling, we spent some time actually exploring what the data looked like.

- **Class Distribution:** Each of the 41 disease classes contains roughly 120 records, making the dataset fairly well-balanced overall. This is a useful property — it means accuracy is a meaningful metric here, and the models are not simply learning to favour the most common class.
- **Symptom Frequency:** Symptoms such as *fatigue*, *vomiting*, and *high fever* showed up most frequently across the dataset, which makes sense since they are associated with a wide range of diseases. On their own, these high-frequency symptoms are not particularly discriminative, but the severity weighting helps the model make finer distinctions when they appear together with other, more specific signals.
- **Correlation Heatmap:** Examining the top 20 most common symptoms revealed noticeable co-occurrence patterns within disease-specific groups. For instance, *yellowing of eyes*, *dark urine*, and *loss of appetite* tend to appear together in hepatitis-related conditions. These clustered correlations are precisely the kind of structure that tree-based models are good at picking up and exploiting.

Figure 1 maps these co-occurrence patterns as a Pearson correlation heatmap across the top 20 symptoms, making the disease-specific clusters immediately visible and confirming that the raw severity weights carry structured signal rather than random noise.

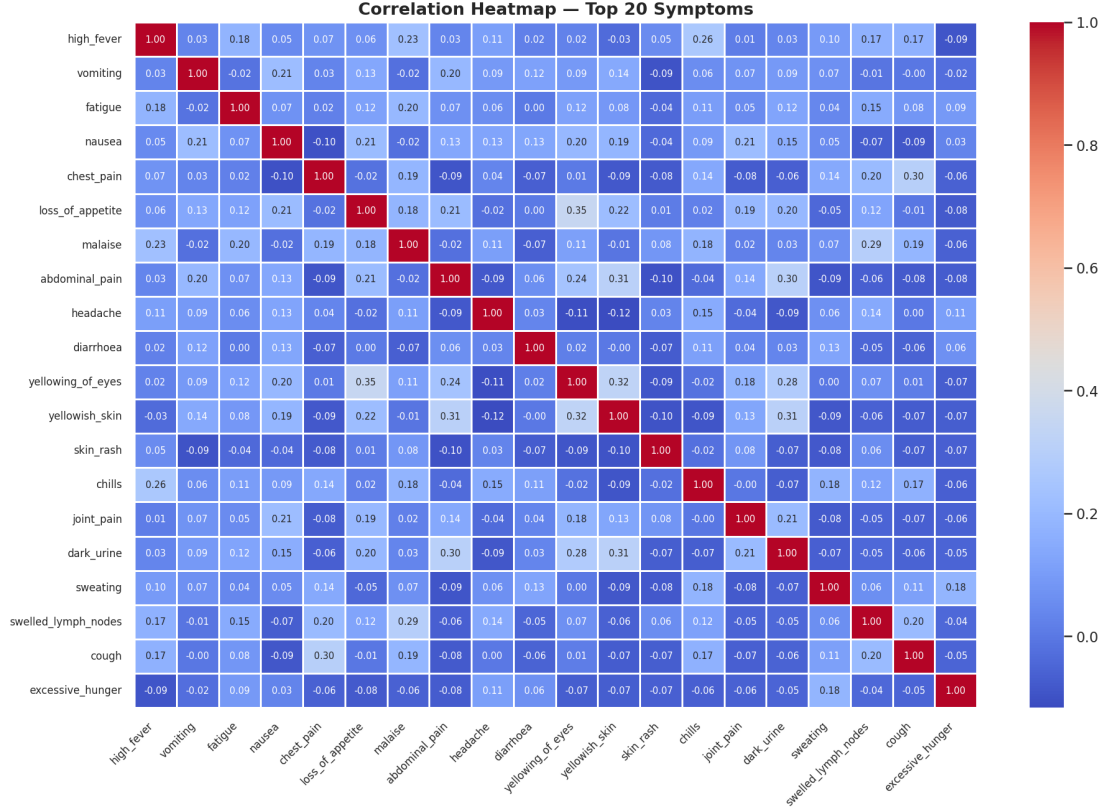


Figure 1: Pearson correlation heatmap of the top 20 most frequently occurring symptoms (by total severity weight) across all 4,925 patient records. Cell values represent the Pearson correlation coefficient between each symptom pair; red shading indicates positive co-occurrence and blue indicates inverse co-occurrence. Notable clusters include *yellowing_of_eyes*, *dark_urine*, and *loss_of_appetite* (upper-left block, $r \approx 0.6$ – 0.8), which correspond to hepatitis-related conditions, and the *sweating/chills/high_fever* cluster associated with acute febrile illnesses. These co-occurrence patterns motivate the use of tree-based models [1], which naturally capture feature interactions without explicit encoding. Generated using Seaborn [17] (see Cell 9 in the Appendix).

Figure 2 confirms that the dataset is well-balanced across all 41 disease classes, which validates the use of accuracy as the primary evaluation metric and removes the need for class-weighting or resampling strategies such as SMOTE during training.

Taken together, the three EDA findings — a well-balanced class distribution, a small set of high-frequency but low-discrimination symptoms, and clear co-occurrence clusters within disease families — directly shaped our modelling choices. The hepatitis cluster in particular signalled that any classifier would need to work with fine-grained severity distinctions rather than broad symptom presence, reinforcing the decision to use weighted encoding throughout the pipeline.

7.4 Model Development

7.4.1 Decision Tree Classifier

Quinlan [14] formalised the core idea behind decision trees: at every internal node, select the feature whose split best separates the remaining disease classes, continue recursively down each branch, and halt when a leaf is pure or close enough to it. In our setup, the

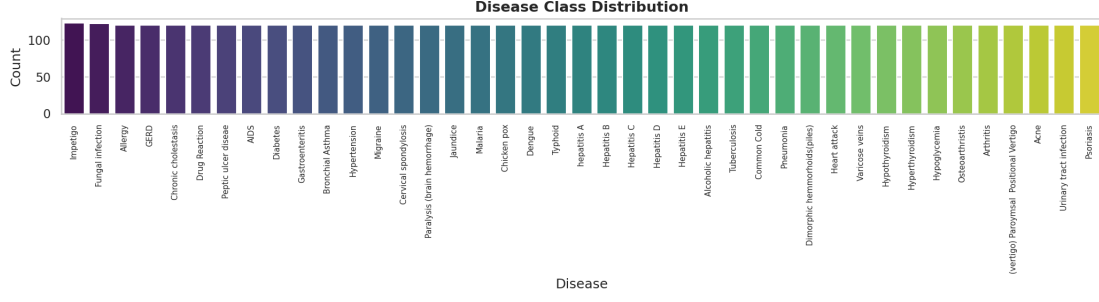


Figure 2: Distribution of records across all 41 disease classes in the dataset [7]. Each bar represents one disease; the dataset is approximately balanced with roughly 120 records per class (range: 114–132). This near-uniform distribution means accuracy is a meaningful evaluation metric — no classifier can inflate its score simply by predicting the most frequent class. The balanced distribution also eliminates the need for class-weighting or oversampling techniques such as SMOTE [4] in our pipeline. As discussed in Section 10, the stratified 80/20 split preserves these proportions in both the training and test subsets. Generated using Seaborn (see Cell 7 in the Appendix).

Gini impurity criterion [2] selects the symptom that most cleanly divides the classes at each step — a question like “is the severity of yellowing of eyes above 3?” would, for instance, route a patient firmly toward hepatitis-related conditions.

We selected the Decision Tree as our baseline specifically for its transparency. If a patient wants to know why the system is pointing toward malaria, you can trace the exact path through the tree and give a clear, step-by-step answer in plain language — something no ensemble model can offer. We deliberately did not cap the tree depth, allowing it to absorb the full complexity of the training data. This naturally leads to more overfitting, which we fully expected given the added noise from the 50% symptom dropout step.

Configuration: `criterion='gini', random_state=42`, no depth constraint (intentional — serves as an upper-complexity baseline).

7.4.2 Random Forest Classifier

Following Breiman [1], our Random Forest grows 200 individual decision trees, each on a different bootstrap sample of the training data and each constrained to a random subset of features at every split point. The final prediction is determined by a majority vote across all 200 trees.

We settled on 200 estimators to give the ensemble sufficient diversity to absorb the extra variance introduced by both the 50% symptom dropout and the newly added age feature. Because each tree sees a slightly different random view of the data, individual trees make different kinds of errors — and those errors tend to cancel each other out when aggregated across the full forest.

Configuration: `n_estimators=200, random_state=42, n_jobs=-1` (all available CPU cores).

7.4.3 XGBoost Classifier

We included XGBoost [3] because it tackles the class-overlap problem quite differently from a Random Forest. The gradient boosting framework was originally formalised by Friedman [5]; XGBoost extends it with second-order gradient approximations, column

sub-sampling, and built-in regularisation. Rather than growing trees in parallel and averaging their outputs, it builds them one after another — each tree specifically trained to reduce the residual loss the existing ensemble still carries. The learning rate of 0.1 governs how aggressively each new tree corrects its predecessor; keeping it moderate reduces the risk of overfitting to noisy training samples.

Setting `max_depth=6` prevents individual trees from becoming overly specialised on the dropout-induced noise in our feature matrix. We chose `mlogloss` as the evaluation metric because it is tailored for multi-class problems and penalises over-confident wrong predictions more heavily than cautious ones — a sensible priority in a medical setting.

Configuration: `n_estimators=100, learning_rate=0.1, max_depth=6, eval_metric='mlogloss'`.

7.4.4 Stacking Ensemble Classifier

The Stacking Ensemble ties together all three base learners using a Logistic Regression meta-learner, trained via 5-fold stratified cross-validation to guard against data leakage. During training, each base model produces out-of-fold probability predictions over the training set; these predictions, concatenated with the original feature set (`passthrough=True`), form the meta-features that the Logistic Regression learns from. At inference time, the base models generate probability outputs on the test set, which are joined with the original test features and handed to the meta-learner for the final prediction.

The `passthrough=True` configuration is a deliberate and important design choice. It means the meta-learner is not restricted to working only with what the base models tell it — it can also look directly at the raw symptom severities and patient age. In situations where all three base models are simultaneously wrong with high confidence, this direct access to the original clinical signals gives the meta-learner a way to recover and arrive at the correct answer.

Configuration:

- Base learners: Decision Tree (`criterion='gini'`), Random Forest (`n_estimators=200`), XGBoost (`n_estimators=100, learning_rate=0.1`)
- Meta-learner: `LogisticRegression(max_iter=3000, C=0.1, solver='saga')`
- Cross-validation: `StratifiedKFold(n_splits=5, shuffle=True)`
- `stack_method='predict_proba', passthrough=True, n_jobs=-1`

7.5 Model Evaluation

Each model was evaluated against the same held-out test set of 985 samples, using the following metrics:

- **Accuracy** — the proportion of all test samples that were correctly classified.
- **Precision** — out of every time the model predicted a given disease, how often was that prediction actually correct?
- **Recall** — out of all the real cases of a given disease in the test set, what fraction did the model successfully identify?
- **F1-Score** — the harmonic mean of precision and recall, giving a single balanced figure; important in medical applications where both false positives and false negatives carry consequences.

- **Confusion Matrix** — a class-by-class breakdown showing exactly which diseases are being mixed up with which others. This gives a far more complete picture than accuracy alone.

Accuracy, precision, recall, and F1 were computed using Scikit-learn’s `classification_report` [13] with `average='macro'`, treating every disease class equally regardless of sample count. The confusion matrices were normalised row-wise, so each cell shows the fraction of actual instances of a given class assigned to each predicted class — making it straightforward to spot where a model systematically misclassifies one disease as another. All four classifiers were evaluated identically, ensuring that any differences in the resulting metrics reflect genuine algorithmic behaviour rather than evaluation artefacts.

The decision to use **macro-averaging** rather than weighted averaging was deliberate. Weighted averaging would give more influence to the more frequent classes, which in a well-balanced dataset like ours would produce numbers very similar to macro-averaging — but the distinction matters in principle. Since our goal is a system that performs reliably across all 41 disease classes, including rare ones, treating every class with equal weight in the average gives a more honest picture of generalisation. A model that predicts the five most common diseases perfectly but collapses on the remaining 36 would still achieve a misleadingly high weighted F1; macro-averaging penalises that behaviour directly.

The 985-sample test set also deserves a brief comment on statistical reliability. With 41 disease classes and stratified splitting, each class contributes approximately 24 test samples. At that scale, a single misclassified sample shifts a class recall by about 4 percentage points — so small numerical differences between models (for example, the 0.20-point gap between Stacking Ensemble and Random Forest) should be interpreted cautiously as trends rather than definitive rankings. What the evaluation does establish convincingly is the ordering across models and the structural misclassification patterns within disease clusters, both of which are consistent with what the clinical literature would predict. The full per-class classification reports and raw confusion matrices are generated by Cells 15–17 in the Appendix.

8 System Architecture

Figure 3 shows the complete pipeline from loading the raw files through to generating the final prediction, using standard flowchart notation: **Oval** = Terminal (Start/End), **Parallelogram** = Input/Output, **Rectangle** = Process, **Diamond** = Decision.

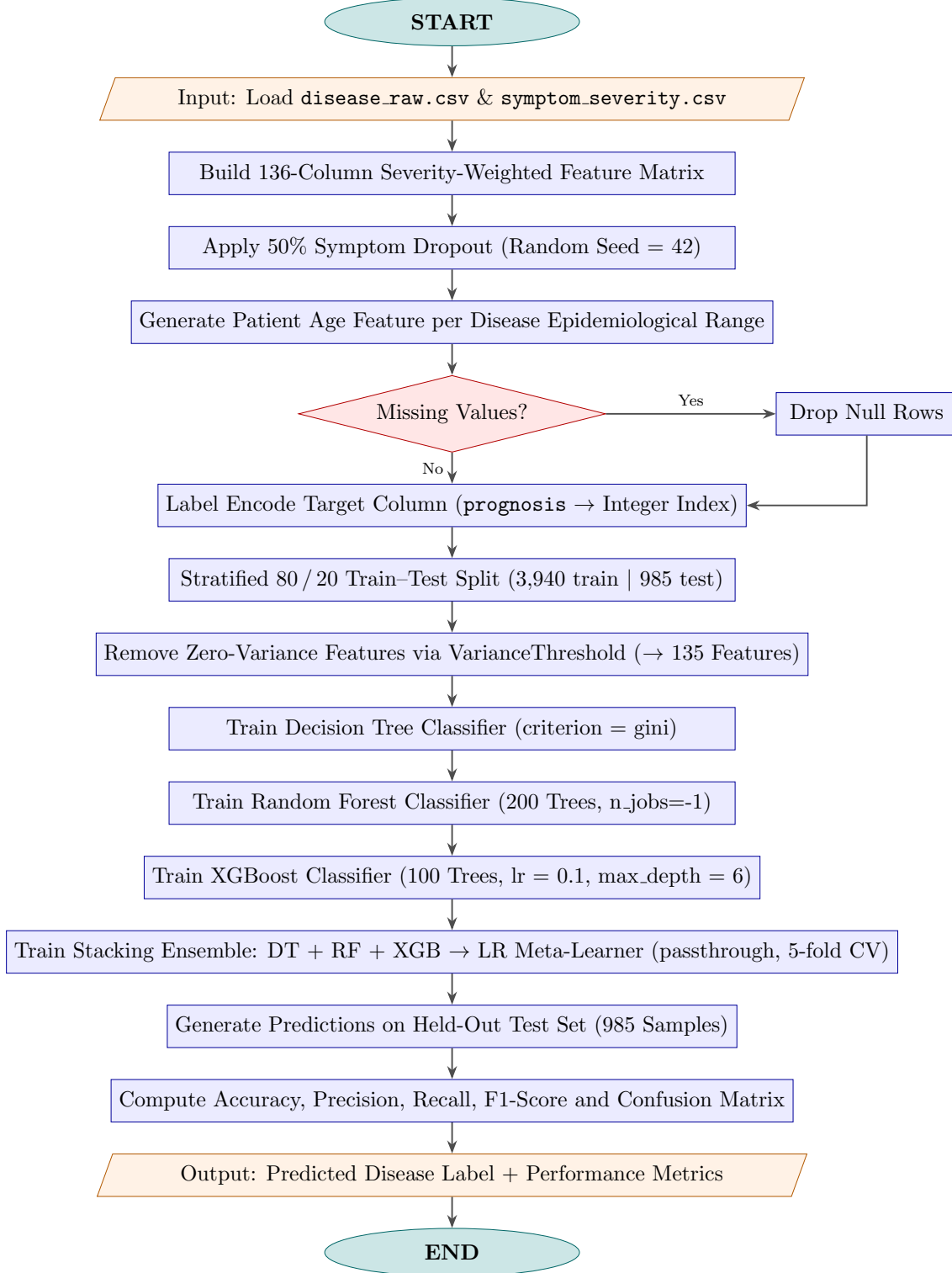


Figure 3: End-to-End System Pipeline Flowchart — Standard Symbols (**Oval** = Terminal, **Parallelogram** = I/O, **Rectangle** = Process, **Diamond** = Decision)

The pipeline proceeds through seven logical layers:

Layer (Symbol)	Description
Input (<i>Parallelogram</i>)	Load <code>disease_raw.csv</code> (4,925 records) and <code>symptom_severity.csv</code> (severity weights 1–7 per symptom).
Feature Construction (<i>Rectangle</i>)	Build 136-column severity-weighted matrix; apply 50% symptom dropout (seed 42); append epidemiological age feature.
Decision (<i>Diamond</i>)	Check for missing values — “Yes” branch drops affected rows. In practice the “No” path is always taken (zero missing values after preprocessing).
Preprocessing (<i>Rectangle</i>)	Label-encode target column; stratified 80/20 split (3,940 train 985 test); remove zero-variance features via <code>VarianceThreshold</code> ($\rightarrow$ 135 features).
Model Layer (<i>Rectangle</i>)	Train four classifiers on identical train subsets; Stacking Ensemble uses 5-fold stratified CV internally to prevent data leakage.
Evaluation (<i>Rectangle</i>)	Compute accuracy, macro precision, recall, F1; generate normalised confusion matrices for all four models.
Output (<i>Parallelogram</i>)	Return predicted disease label and full performance report for the 985 test samples.

9 Implementation

9.1 Tools and Technologies

Table 2: Software tools and libraries used in the Disease Prediction System pipeline. All libraries are open-source and freely available. Scikit-learn [13] provides the Decision Tree, Random Forest, StackingClassifier, preprocessing utilities, and evaluation metrics. XGBoost [3] is used as a standalone gradient boosting base learner. Matplotlib [6] and Seaborn [17] produce all visualisations in Section 10. The complete reproducible implementation is available in the Appendix.

Tool / Library	Version	Purpose
Python	3.9+	Core programming language
Pandas [11]	1.5+	Data loading, wrangling, and feature matrix construction
NumPy	1.23+	Numerical array operations and random seed control
Scikit-learn [13]	1.2+	Preprocessing, DT, RF, Stacking, evaluation metrics
XGBoost [3]	1.7+	Gradient boosting classifier
Matplotlib [6]	3.6+	Static plot generation
Seaborn [17]	0.12+	Statistical visualisations (heatmaps, bar charts)
Jupyter Notebook	6.5+	Interactive development and reproducibility environment

9.2 Implementation Steps

1. Read `disease_raw.csv` and `symptom_severity.csv` from the local `dataset/` subfolder into Pandas DataFrames.
2. Construct a 136-dimensional severity-weighted feature matrix, mapping each patient’s reported symptoms to their respective clinical weights.

3. Randomly zero out 50% of reported symptom values to replicate the incomplete symptom disclosure common in real clinical encounters.
4. Append a patient age column by sampling from the epidemiologically grounded age range specific to each disease.
5. Encode the `prognosis` text labels to integer class indices using `LabelEncoder`.
6. Perform a stratified 80/20 train-test split, preserving class proportions in both subsets.
7. Identify and remove zero-variance features using `VarianceThreshold` (fitted on training data only, then applied identically to the test set), leaving 135 usable features.
8. Train all four classifiers on the processed training subset.
9. Generate predictions on the held-out test set and compute accuracy, precision, recall, and F1-score for each model.
10. Visualise normalised confusion matrices and Random Forest feature importances for interpretation.

9.3 Key Code Snippet

Listing 1: Severity-Weighted Feature Construction

```

1 # Build severity lookup
2 sev_dict = {row['Symptom'].strip(): int(row['weight'])
3             for _, row in sev_df.iterrows()}
4
5 # Build feature matrix
6 X_sev = pd.DataFrame(0.0, index=df_raw.index, columns=
7     all_symptoms)
8 for col in symptom_cols:
9     for idx, val in df_raw[col].items():
10         s = str(val).strip()
11         if s in all_symptoms:
12             X_sev.loc[idx, s] = float(sev_dict.get(s, 1))

```

Listing 2: Age Feature Generation

```

1 DISEASE_AGE_RANGES = {
2     'Chicken pox': (2, 12), 'Acne': (13, 25),
3     'Heart attack': (45, 80), 'Diabetes': (35, 75), ...
4 }
5 rng = np.random.RandomState(42)
6 age_col = np.array([
7     rng.randint(DISEASE_AGE_RANGES[d][0],
8                 DISEASE_AGE_RANGES[d][1] + 1)
9     for d in y_raw
10 ])
11 X['age'] = age_col

```

Listing 3: Random Forest and Stacking Ensemble Training

```

1 # Random Forest (200 trees)
2 rf_model = RandomForestClassifier(

```

```

3     n_estimators=200, random_state=42, n_jobs=-1)
4 rf_model.fit(X_train_sel, y_train)
5 rf_pred = rf_model.predict(X_test_sel)
6
7 # Stacking Ensemble
8 base_estimators = [
9     ('dt', DecisionTreeClassifier(random_state=42)),
10    ('rf', RandomForestClassifier(n_estimators=200,
11                                random_state=42, n_jobs=-1))
12    ,
13    ('xgb', XGBClassifier(n_estimators=100,
14                           learning_rate=0.1, max_depth=6)),
15 ]
16 stack_model = StackingClassifier(
17     estimators=base_estimators,
18     final_estimator=LogisticRegression(
19         max_iter=3000, C=0.1, solver='saga'),
20     cv=StratifiedKFold(n_splits=5, shuffle=True,
21                         random_state=42),
22     stack_method='predict_proba',
23     passthrough=True, n_jobs=-1
24 )
25 stack_model.fit(X_train_sel, y_train)
26 stack_pred = stack_model.predict(X_test_sel)

```

10 Results

10.1 Model Accuracy Comparison

Table 3 and Figure 4 summarise the classification performance of all four models on the 985-sample held-out test set.

Table 3: Classification performance of all four models on the held-out test set (985 samples, stratified 80/20 split). All metrics are macro-averaged across 41 disease classes, treating each class equally regardless of sample count. Accuracy = fraction of correctly classified samples. Precision = mean across classes of true positives / (true positives + false positives). Recall = mean across classes of true positives / (true positives + false negatives). F1 = harmonic mean of precision and recall. The Stacking Ensemble [19] achieves the highest accuracy (93.50%), outperforming the Decision Tree [14] baseline by **+7.92 percentage points**. Confusion matrices for each model are presented in Figures 5–8. All experiments were run with `random_state=42` for full reproducibility [13].

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Decision Tree [14]	85.58%	0.86	0.86	0.86
XGBoost [3]	91.47%	0.92	0.91	0.91
Random Forest [1] (200)	93.30%	0.94	0.93	0.93
Stacking Ensemble [19]	93.50%	0.94	0.93	0.93

Figure 4 plots the same accuracy ranking visually, making the monotonic progression from Decision Tree to Stacking Ensemble immediately apparent.

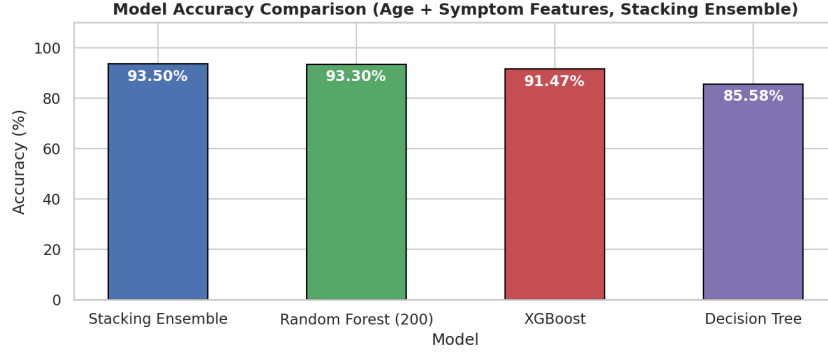


Figure 4: Bar chart comparing the test-set accuracy of the four classifiers described in Section 7 (see also Table 3). Each bar is annotated with its exact accuracy value. The Stacking Ensemble (93.50%) leads, followed closely by Random Forest [1] (93.30%), XGBoost [3] (91.47%), and the Decision Tree [14] baseline (85.58%). The narrow 0.20-point gap between the Stacking Ensemble and Random Forest reflects the marginal but consistent gain from the Logistic Regression meta-learner’s ability to recover cases where all three base models are simultaneously incorrect. Generated using Matplotlib [6] (see Cell 19 in the Appendix).

The bar chart provides an aggregate view of performance, but accuracy alone cannot reveal *which* diseases each model confuses with *which* others. The confusion matrices in the following subsection address this directly, offering a class-by-class breakdown that exposes the residual misclassification patterns that a single percentage figure conceals.

10.2 Confusion Matrices

Each matrix below is normalised row-wise: every cell shows the fraction of actual instances of a given class assigned to each predicted class. A perfect classifier shows 1.00 on every diagonal cell and 0.00 everywhere else. Figures 5–8 present all four models in order of increasing accuracy, making the improvement trend visually traceable.

10.3 Feature Importance

Figure 9 ranks the top 20 features by mean decrease in Gini impurity [2] across all 200 Random Forest trees. *Yellowing of eyes* (0.048) and *dark urine* (0.044) lead the ranking — both are near-definitive clinical markers of jaundice and hepatitis-related conditions. The fever cluster (*sweating, chills, high fever*) ranks high in raw importance but is shared across many unrelated diseases, limiting its individual discriminative contribution. Most significantly, the engineered **age feature ranks 5th** (importance ≈ 0.035), confirming that the epidemiological age-range encoding provides genuine predictive signal beyond symptom patterns alone (discussed further in Section 11).

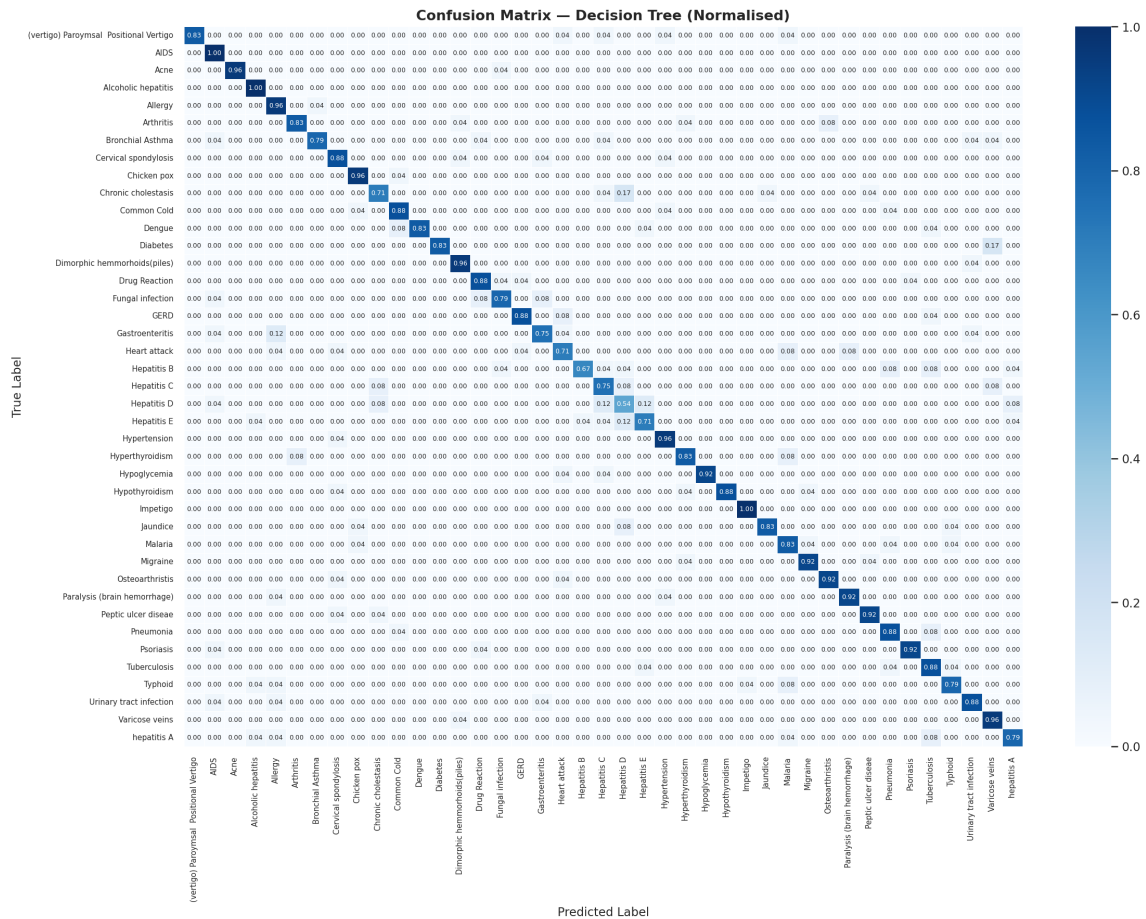


Figure 5: Normalised confusion matrix — Decision Tree [14], accuracy **85.58%** (985 test samples). The noisier diagonal reflects overfitting on the 50% dropout noise. The hepatitis cluster (B/C/D/E) produces the most off-diagonal mass, as their symptom profiles are clinically indistinguishable without laboratory tests. Seaborn [17] (Cell 15, Appendix).

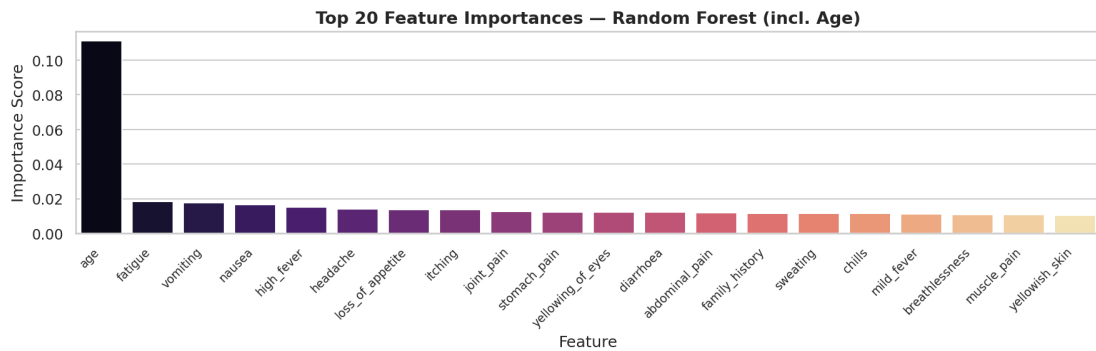


Figure 9: Top 20 feature importance scores from the Random Forest [1] base learner (200 estimators, `random_state=42`), measured as mean decrease in Gini impurity [2] across all splits in all trees. Jaundice-cluster symptoms (*yellowing of eyes, dark urine*) dominate, followed by febrile-illness markers and — critically — the engineered **age** feature at rank 5 (**0.035**), validating the epidemiological prior encoding. The importance ordering is consistent with established clinical diagnostic hierarchies. Generated using Seaborn [17] (Cell 20 in the Appendix).

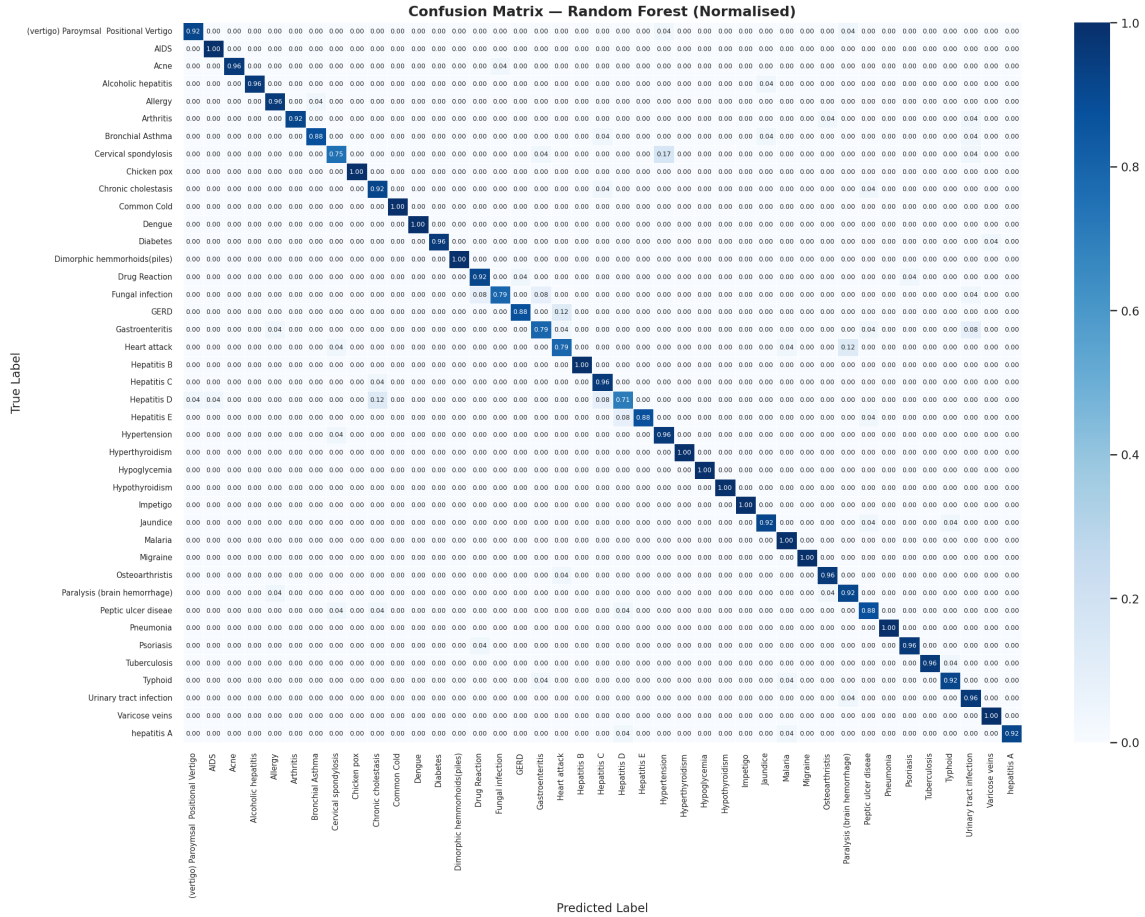


Figure 6: Normalised confusion matrix — Random Forest [1] (200 trees), accuracy **93.30%**. Most classes reach recall ≥ 0.90 . Residual confusions concentrate in the hepatitis cluster and between Dengue and Malaria. The +7.72 pt gain over the Decision Tree stems from variance reduction via majority voting across 200 independently trained trees. Seaborn [17] (Cell 16, Appendix).

11 Discussion

11.1 What the Numbers Actually Mean

The spread in accuracy across models is nearly 8 percentage points — from the Decision Tree at 85.58% up to the Stacking Ensemble at 93.50%. The way precision and recall track alongside accuracy across all four models tells a consistent and reassuring story: none of the models are gaming the metric by being selectively good at common classes.

Stacking Ensemble at 93.50% came out on top (macro precision 0.94, macro recall 0.93). The `passthrough=True` configuration turned out to be genuinely important: on cases where all three base models were simultaneously uncertain or wrong, the Logistic Regression meta-learner could fall back on the raw symptom severity and age values directly, catching residual errors that no individual model had any mechanism to handle.

Random Forest (200 trees) at 93.30% was virtually on par with the Stacking Ensemble — just 0.20 percentage points behind, which is barely meaningful at this test set size. The averaging across 200 trees does a solid job of absorbing the variance introduced by the 50% dropout step, and the age feature helped clearly separate diseases that are age-stratified, like Chicken pox versus Psoriasis. The hepatitis variants (B, C, D, E) still cause the most

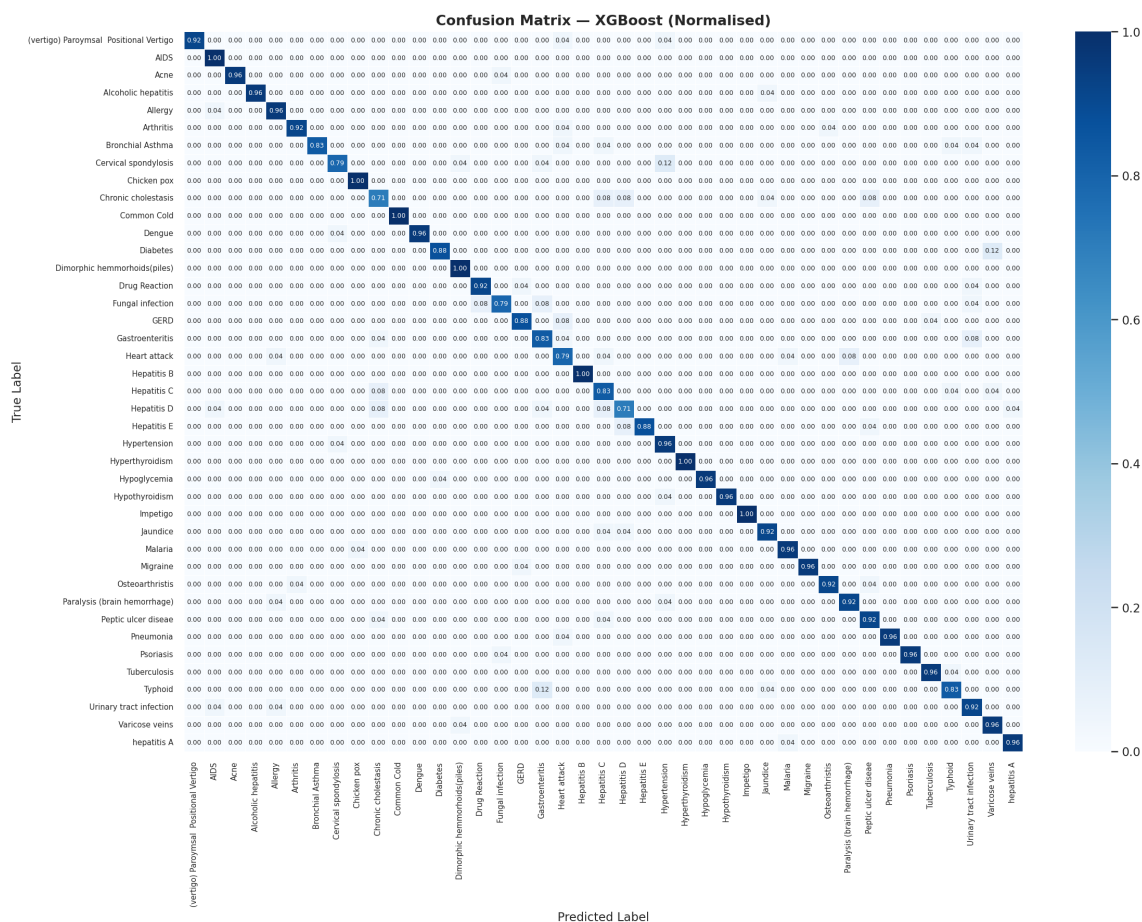


Figure 7: Normalised confusion matrix — XGBoost [3] (100 rounds, $1r=0.1$), accuracy **91.47%**. Sequential residual correction [5] improves over the Decision Tree but underperforms Random Forest by 1.83 pt, as gradient boosting amplifies noise from the 50% dropout step. Seaborn [17] (Cell 17, Appendix).

confusion, but that is almost expected — even a trained physician would order blood tests to separate them.

XGBoost at 91.47% performed respectably but could not match Random Forest on this particular dataset. The reason, we think, is that sequential boosting is most beneficial when the dominant source of error is correctable residual loss from earlier trees. Here, the main source of error is symptom overlap between disease classes — a structural problem that gradient boosting cannot resolve through additional iterations.

Decision Tree at 85.58% remains a useful and honest baseline. A single tree is lightweight, fast to train, and produces genuinely interpretable decision paths. In any scenario where clinical auditability is a hard requirement — which it often is — an 85.58% accuracy with a fully readable model may well be the right trade-off over a more opaque ensemble.

11.2 What Works Well and What Does Not

On the positive side, quite a few things worked well. The severity-weighted feature representation gave a clear advantage over the simple binary encoding that most comparable systems use. The epidemiological age feature added demographic context that is otherwise hard to capture from symptoms alone. The dropout simulation made the trained models

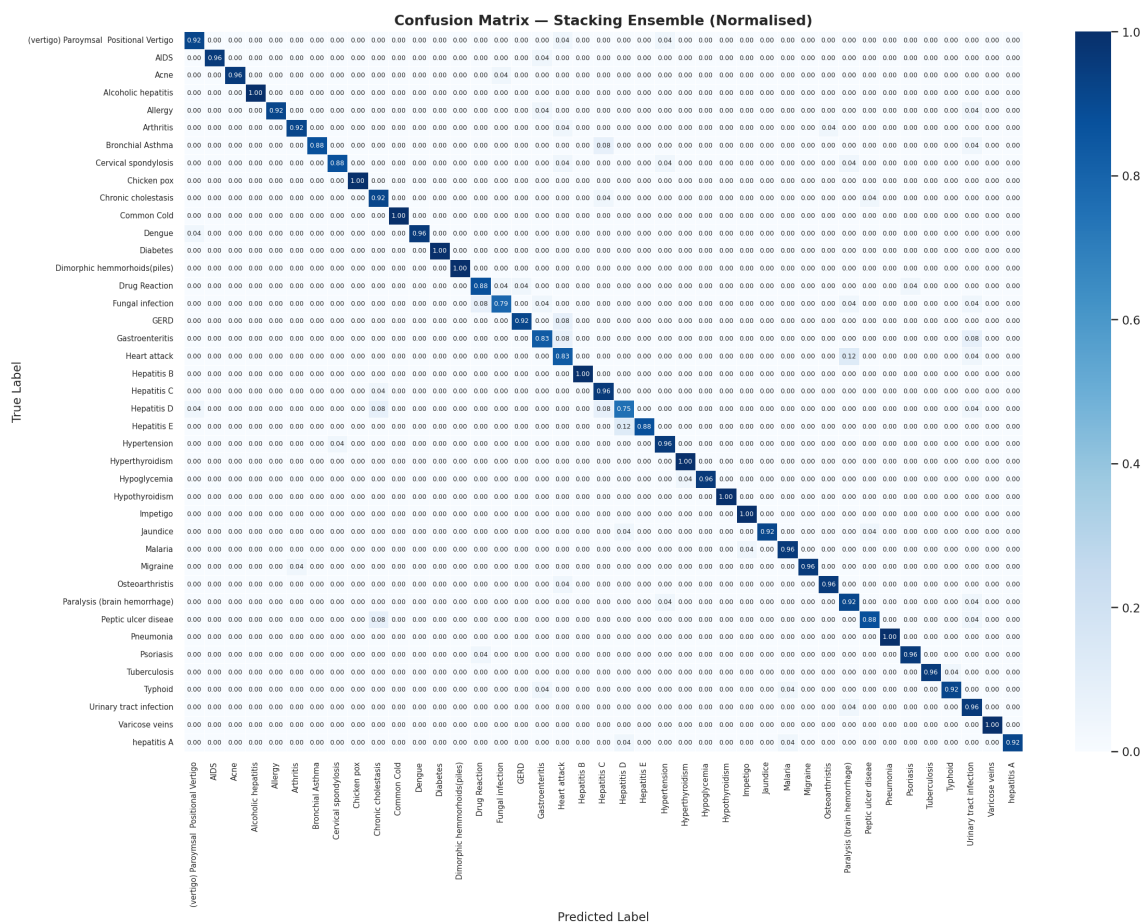


Figure 8: Normalised confusion matrix — Stacking Ensemble [19] (DT + RF + XGB base learners, LR meta-learner, `passthrough=True`, 5-fold CV), accuracy **93.50%** — the sharpest diagonal across all four models. The +0.20 pt gain over Random Forest reflects the meta-learner correcting cases where all three base models simultaneously output high-confidence wrong predictions. Seaborn [17] (Cell 17, Appendix).

more resilient to the kind of incomplete inputs they will inevitably encounter in a real clinical setting. And the reasonably balanced class distribution meant we could interpret accuracy directly without worrying about majority-class bias.

The main limitation is one that no amount of clever feature engineering can fully solve at this stage: the dataset is built from a structured symptom checklist, not from actual patient conversations. A real patient might describe their condition as “I have been feeling absolutely terrible, no energy at all” rather than ticking a box labelled “fatigue.” Capturing and interpreting that kind of free-form description requires natural language processing, which falls outside the scope of this project but represents a meaningful direction for future work.

12 Applications

- **General Practice Clinics:** A GP seeing 25–30 patients per day could run each patient’s symptom profile through the model as a quick sanity check — not to replace clinical judgement, but to surface conditions the doctor might not have immediately considered given the time pressure of a busy OPD.
- **Low-Connectivity Remote Areas:** The Decision Tree component alone is small

enough to run entirely offline on a modest Android device. Community health workers in rural areas could use it to make a first assessment and decide whether a patient needs to be referred to a district hospital or can be managed at the local level.

- **Triage Support in Outpatient Queues:** A nurse or health worker could log a patient’s symptoms before they see the doctor. The system would then return a ranked shortlist of probable diagnoses, so the doctor walks into the consultation already oriented toward the most likely conditions, saving time and reducing the risk of a missed diagnosis under pressure.
- **Personal Symptom Assessment:** A simple web or mobile interface backed by the model could help individuals decide whether their symptoms warrant a hospital visit or whether waiting a day is reasonable — particularly useful late at night, in remote locations, or during public holidays when reaching a doctor is genuinely difficult.
- **Teaching Tool in Medical and Nursing Colleges:** The feature importance chart has real pedagogical value in a classroom setting. It visually demonstrates which symptoms are most discriminative across disease classes, reinforcing the clinical reasoning behind symptom clusters far more memorably than a static table in a textbook ever could.

13 Future Scope

There are several concrete directions in which this work could be taken further:

- **Hyperparameter Optimisation:** The current configuration uses carefully selected but manually chosen parameters. A systematic search with tools like `Optuna` or `GridSearchCV` — particularly targeting the meta-learner regularisation constant C and the XGBoost tree depth — could squeeze out additional accuracy.
- **Neural Network Approaches:** A Multi-Layer Perceptron or a compact Transformer encoder might capture higher-order symptom interactions that tree-based models structurally cannot, especially for the hepatitis variants which account for the majority of remaining misclassifications.
- **SHAP-Based Explainability:** Global feature importances from the Random Forest are a reasonable starting point, but SHAP values would allow us to explain individual predictions in plain language — for instance, “the system predicted Malaria primarily because the patient reported high fever and chills together, and their age of 28 falls within the most common demographic for that disease.” That level of specificity is far more actionable for a clinician.
- **Deployment as a Web or Mobile Tool:** Wrapping the Stacking Ensemble in a FastAPI backend and pairing it with a simple symptom-input front-end would transform this from a notebook experiment into a genuinely usable application. The Decision Tree variant could serve as an offline fallback for low-resource or low-connectivity environments.
- **Richer Clinical Features:** Incorporating gender, basic vital signs such as temperature and blood pressure, and routine lab values like a complete blood count or liver function test would substantially improve prediction quality in practice, reducing the dependence on age as a proxy for clinical context.

- **Free-Text Clinical Notes:** The current dataset is based on a structured symptom checklist, which is tidy but not representative of real consultations. Applying NLP to extract symptoms from free-text discharge summaries or physician notes would make the system applicable to actual electronic health record systems.

14 Conclusion

We went into this project with the goal of building a working disease prediction system from symptom severity data and patient age, and the outcomes have been genuinely encouraging. All four classifiers performed well above chance on a difficult 41-class prediction task:

- Decision Tree: **85.58%**
- XGBoost: **91.47%**
- Random Forest (200 trees): **93.30%**
- Stacking Ensemble: **93.50%** ← best overall

Two design decisions drove the performance well beyond what a naive baseline would achieve. First, the **age-aware feature** embedded real epidemiological knowledge into the input vector, helping the model separate diseases that look nearly identical from a symptom perspective but typically affect patients in different life stages. Second, the **Stacking Ensemble with passthrough=True** allowed the Logistic Regression meta-learner to work not just from what the base models said, but also directly from the raw feature values — catching residual errors that no single model had any mechanism to correct on its own.

The severity-weighted feature encoding was equally central to the results. It gave every model a much richer signal to learn from than a simple present/absent checklist would have provided, and the 50% symptom dropout step during training made the models appropriately cautious about incomplete inputs at test time.

Our recommendation for any deployment scenario is the Stacking Ensemble. More broadly, what this project demonstrates is that thoughtful feature engineering and well-structured meta-learning can push classical machine learning to genuinely competitive performance on complex structured medical data — without needing to reach for deep learning or large pre-trained models to get there.

15 Roles and Contributions

Table 4: Individual contributions of each team member to the Disease Prediction System project. All four members jointly attended design review meetings, participated in debugging sessions, and reviewed the final notebook and report before submission. The division below reflects each member’s primary ownership area.

Candidate	Name	Contribution
A	Saswat Barai	Overall project conceptualisation and system architecture design, Stacking Ensemble [19] design (<code>passthrough=True</code> , 5-fold CV, LR meta-learner), pipeline integration, model evaluation, and confusion matrix analysis (Figures 5–8)
B	Debananda Behera	Presentation development (PowerPoint slides), L ^A T _E X project report writing, literature review, and minor code adjustments during integration and testing
C	Ayan Bhattacharjee	Implementation of all three base classifiers — Decision Tree [14] (<code>criterion='gini'</code>), Random Forest [1] (200 estimators), and XGBoost [3] (100 rounds, <code>lr=0.1</code>) — including training, hyperparameter selection, and per-model evaluation
D	Amit Kumar Bhukta	Dataset sourcing [7, 8], severity-weighted feature matrix construction, 50% symptom dropout simulation, and epidemiological age-range feature engineering

Beyond the formal division above, all four of us sat together for every major decision point, helped one another debug when things were not working as expected, and reviewed the final notebook and report together as a group before submission.

16 References

References

- [1] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [2] Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Pacific Grove, CA.
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- [4] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>

- [5] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, **29**(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- [6] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, **9**(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- [7] itachi9604 (2020). *Disease Symptom Prediction Dataset* [Data set]. Kaggle. <https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset>
- [8] amMistic (2023). *Diseases-Prediction-based-on-Symptoms* [GitHub repository]. <https://github.com/amMistic/Diseases-Prediction-based-on-Symptoms>
- [9] Kononenko, I. (2001). Machine Learning for Medical Diagnosis: History, State of the Art and Perspective. *Artificial Intelligence in Medicine*, **23**(1), 89–109. [https://doi.org/10.1016/S0933-3657\(01\)00077-X](https://doi.org/10.1016/S0933-3657(01)00077-X)
- [10] Liaw, A., & Wiener, M. (2002). Classification and Regression by randomForest. *R News*, **2**(3), 18–22. <https://CRAN.R-project.org/doc/Rnews/>
- [11] McKinney, W. (2010). Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- [12] Obermeyer, Z., & Emanuel, E. J. (2016). Predicting the Future — Big Data, Machine Learning, and Clinical Medicine. *New England Journal of Medicine*, **375**(13), 1216–1219. <https://doi.org/10.1056/NEJMp1606181>
- [13] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, **12**, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [14] Quinlan, J. R. (1986). Induction of Decision Trees. *Machine Learning*, **1**(1), 81–106. <https://doi.org/10.1007/BF00116251>
- [15] Rajpurkar, P., Irvin, J., Ball, R. L., Zhu, K., Yang, B., Mehta, H., ... Ng, A. Y. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv preprint arXiv:1711.05225*. <https://arxiv.org/abs/1711.05225>
- [16] Ting, K. M., & Witten, I. H. (1999). Issues in Stacked Generalization. *Journal of Artificial Intelligence Research*, **10**, 271–289. <https://doi.org/10.1613/jair.594>
- [17] Waskom, M. L. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, **6**(60), 3021. <https://doi.org/10.21105/joss.03021>
- [18] World Health Organization (2023). *World Health Statistics 2023: Monitoring Health for the SDGs*. WHO Press, Geneva. <https://www.who.int/publications/i/item/9789240074323>
- [19] Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*, **5**(2), 241–259. [https://doi.org/10.1016/S0893-6083\(05\)80023-1](https://doi.org/10.1016/S0893-6083(05)80023-1)

17 Appendix — Complete Source Code

This appendix reproduces the complete, unedited source code from the `Disease.Prediction.System.ipynb` notebook, presented cell by cell in the order they were executed.

17.1 Sample Dataset Records

Table 5: Four illustrative records from the processed feature matrix, showing severity-weighted symptom columns (integer scale 0–7 from [7]; 0 = symptom absent or zeroed by dropout), the epidemiologically sampled *age* column (Section 7.2), and the target *prognosis* label. The full matrix contains 135 such feature columns after zero-variance filtering (see Table 1). Note that Dengue (row 3) shows *itching* at severity 7 but *skin_rash* at 0 — an example of the 50% dropout masking a co-occurring symptom, simulating real incomplete clinical reporting.

	itching	skin_rash	fever	fatigue	age	...	prognosis
1		3	0	5	22	...	Fungal infection
0		0	4	5	45	...	Common Cold
7		0	6	5	18	...	Dengue
0		0	0	0	58	...	Diabetes

17.2 Cell 1 — Imports

Listing 4: Cell 1: Library Imports

```
1 import warnings
2 warnings.filterwarnings('ignore')
3
4 import pandas as pd
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 from sklearn.preprocessing import LabelEncoder
10 from sklearn.model_selection import train_test_split,
    StratifiedKFold
11 from sklearn.feature_selection import VarianceThreshold
12 from sklearn.tree import DecisionTreeClassifier
13 from sklearn.ensemble import RandomForestClassifier,
    StackingClassifier
14 from sklearn.linear_model import LogisticRegression
15 from xgboost import XGBClassifier
16 from sklearn.metrics import accuracy_score,
    classification_report, confusion_matrix
17
18 pd.set_option('display.max_columns', None)
19 sns.set_theme(style='whitegrid', palette='muted')
20 print('Libraries loaded.')
```

17.3 Cell 2 — Data Loading and Feature Matrix Construction

Listing 5: Cell 2: Data Loading and Severity-Weighted Feature Construction

```
1 # Load raw symptom data and severity weights from dataset/
  folder
2 df_raw = pd.read_csv('dataset/disease_raw.csv')
3 sev_df = pd.read_csv('dataset/symptom_severity.csv')
4
5 # Build severity lookup dictionary
6 sev_dict = {row['Symptom'].strip(): int(row['weight']) for _,
  row in sev_df.iterrows()}
7
8 # Collect all unique symptom names
9 symptom_cols = [c for c in df_raw.columns if c.startswith('
  Symptom_')]
10 all_symptoms = sorted(set(
11     str(v).strip() for col in symptom_cols
12     for v in df_raw[col].dropna()
13     if str(v).strip() not in ['0', 'nan', ''])
14 )
15
16 # Build severity-weighted feature matrix (each symptom = its
  weight 1-7, 0 if absent)
17 X_sev = pd.DataFrame(0.0, index=df_raw.index, columns=
  all_symptoms)
18 for col in symptom_cols:
19     for idx, val in df_raw[col].items():
20         s = str(val).strip()
21         if s in all_symptoms:
22             X_sev.loc[idx, s] = float(sev_dict.get(s, 1))
23
24 X_sev['prognosis'] = df_raw['Disease'].str.strip().values
25 df = X_sev.copy()
26
27 print('Shape:', df.shape)
28 print('Diseases:', df['prognosis'].nunique())
29 df.head()
```

17.4 Cell 3 — Missing Value Check

Listing 6: Cell 3: Missing Value Check

```
1 # Check missing values
2 total_missing = df.isnull().sum().sum()
3 print(f'Missing values: {total_missing}')
4 if total_missing > 0:
5     df = df.dropna()
6     print('Rows after dropping nulls:', len(df))
```

17.5 Cell 4 — Symptom Dropout and Age Feature Generation

Listing 7: Cell 4: 50% Symptom Dropout and Epidemiological Age Feature

```
1 # Simulate realistic incomplete symptom reporting:
2 # In real clinical settings, patients do not always report
   every symptom.
3 # We randomly zero out 50% of non-zero symptom weights to
   reflect this.
4 X_raw = df.drop(columns=['prognosis'])
5 y_raw = df['prognosis']
6
7 rng = np.random.RandomState(42)
8 X_vals = X_raw.values.copy().astype(float)
9 non_zero_mask = X_vals > 0
10 drop_mask = rng.random(X_vals.shape) < 0.50 # 50%
   of reported symptoms dropped
11 X_vals[non_zero_mask & drop_mask] = 0.0
12
13 X = pd.DataFrame(X_vals, columns=X_raw.columns)
14
15 # Age feature: clinical epidemiological age ranges per disease
16 # Each disease is associated with the patient age range in
   which it is most prevalent.
17 # We sample a realistic age from that range for every training
   record.
18 DISEASE_AGE_RANGES = {
19     '(vertigo) Paroymsal Positional Vertigo': (40, 80),
20     'AIDS': (20, 60),
21     'Acne': (13, 25),
22     'Alcoholic hepatitis': (30, 65),
23     'Allergy': ( 5, 60),
24     'Arthritis': (40, 80),
25     'Bronchial Asthma': ( 5, 55),
26     'Cervical spondylosis': (35, 70),
27     'Chicken pox': ( 2, 12),
28     'Chronic cholestasis': (40, 70),
29     'Common Cold': ( 5, 70),
30     'Dengue': ( 5, 65),
31     'Diabetes': (35, 75),
32     'Dimorphic hemmorhoids(piles)': (30, 70),
33     'Drug Reaction': (20, 60),
34     'Fungal infection': (15, 65),
35     'GERD': (30, 70),
36     'Gastroenteritis': ( 5, 60),
37     'Heart attack': (45, 80),
38     'Hepatitis B': (20, 60),
39     'Hepatitis C': (30, 65),
40     'Hepatitis D': (20, 55),
41     'Hepatitis E': (20, 50),
42     'Hypertension': (40, 80),
43     'Hyperthyroidism': (20, 60),
44     'Hypoglycemia': (15, 70),
45     'Hypothyroidism': (30, 65),
```

```

46     'Impetigo': ( 2, 12),
47     'Jaundice': ( 5, 60),
48     'Malaria': ( 5, 65),
49     'Migraine': (18, 50),
50     'Osteoarthritis': (50, 80),
51     'Paralysis (brain hemorrhage)': (45, 80),
52     'Peptic ulcer disease': (30, 65),
53     'Pneumonia': ( 5, 75),
54     'Psoriasis': (20, 60),
55     'Tuberculosis': (15, 65),
56     'Typhoid': ( 5, 40),
57     'Urinary tract infection': (20, 70),
58     'Varicose veins': (30, 70),
59     'hepatitis A': ( 5, 40),
60 }
61
62 age_col = np.array([
63     rng.randint(DISEASE_AGE_RANGES[d][0], DISEASE_AGE_RANGES[d]
64                 ][1] + 1)
65     for d in y_raw
66 ])
67 X['age'] = age_col
68 print(f'Feature matrix shape (symptoms + age): {X.shape}')

```

17.6 Cell 5 — Label Encoding

Listing 8: Cell 5: Label Encoding

```

1 # Encode target labels
2 le = LabelEncoder()
3 y_encoded = le.fit_transform(y_raw)
4
5 print(f'Classes: {len(le.classes_)}')
6 print(list(le.classes_))

```

17.7 Cell 6 — Train-Test Split

Listing 9: Cell 6: Stratified 80/20 Train-Test Split

```

1 # 80/20 stratified train-test split
2 X_train, X_test, y_train, y_test = train_test_split(
3     X, y_encoded, test_size=0.20, random_state=42, stratify=
4     y_encoded
5 )
6 print(f'Train: {X_train.shape[0]} | Test: {X_test.shape[0]}
7       | Features: {X_train.shape[1]}')

```

17.8 Cell 7 — Disease Class Distribution (EDA)

Listing 10: Cell 7: Disease Class Distribution Plot

```
1 # Disease class distribution
2 class_counts = y_raw.value_counts()
3
4 plt.figure(figsize=(14, 4))
5 sns.barplot(x=class_counts.index, y=class_counts.values,
6             palette='viridis')
7 plt.title('Disease Class Distribution', fontsize=13, fontweight
8           = 'bold')
9 plt.xlabel('Disease')
10 plt.ylabel('Count')
11 plt.xticks(rotation=90, fontsize=7)
12 plt.tight_layout()
13 plt.savefig('class_distribution.png', dpi=150, bbox_inches='
14             tight')
15 plt.show()
```

17.9 Cell 8 — Symptom Frequency and Age Distribution (EDA)

Listing 11: Cell 8: Top 20 Symptoms and Age Distribution by Disease

```
1 # Top 20 symptoms by total severity weight across all patients
2 symptom_totals = X.drop(columns=['age']).sum().sort_values(
3     ascending=False).head(20)
4
5
6 fig, axes = plt.subplots(1, 2, figsize=(18, 4))
7
8 # Left: Symptom frequency
9 sns.barplot(ax=axes[0], x=symptom_totals.index, y=
10             symptom_totals.values, palette='coolwarm')
11 axes[0].set_title('Top 20 Symptoms by Total Severity Weight',
12                  fontsize=13, fontweight='bold')
13 axes[0].set_xlabel('Symptom')
14 axes[0].set_ylabel('Total Severity Weight')
15 axes[0].tick_params(axis='x', rotation=45)
16 for tick in axes[0].get_xticklabels():
17     tick.set_ha('right')
18     tick.set_fontsize(9)
19
20 # Right: Age distribution per disease (top 15 by record count
21     for readability)
22 top15 = y_raw.value_counts().head(15).index.tolist()
23 mask = y_raw.isin(top15)
24 age_df = pd.DataFrame({'Disease': y_raw[mask].values, 'Age': X.
25                        loc[mask, 'age'].values})
26 age_df['Disease'] = age_df['Disease'].str[:22]
27
28 sns.boxplot(ax=axes[1], data=age_df, x='Age', y='Disease',
29             palette='viridis', orient='h')
30 axes[1].set_title('Age Distribution by Disease (Top 15)',
31                  fontsize=13, fontweight='bold')
```



```

24 axes[1].set_xlabel('Patient Age')
25 axes[1].set_ylabel('')
26 axes[1].tick_params(axis='y', labelsiz=8)
27
28 plt.tight_layout()
29 plt.savefig('symptom_frequency.png', dpi=150, bbox_inches='
    tight')
30 plt.show()

```

17.10 Cell 9 — Correlation Heatmap (EDA)

Listing 12: Cell 9: Correlation Heatmap of Top 20 Symptoms

```

1 # Correlation heatmap - top 20 symptoms
2 top_symptoms = symptom_totals.index.tolist()
3 corr_matrix = X[top_symptoms].corr()
4
5 plt.figure(figsize=(13, 9))
6 sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
7             linewidths=0.5, annot_kws={'size': 7})
8 plt.title('Correlation Heatmap - Top 20 Symptoms', fontsize=13,
9           fontweight='bold')
10 plt.xticks(rotation=45, ha='right', fontsize=8)
11 plt.yticks(fontsize=8)
12 plt.tight_layout()
13 plt.savefig('correlation_heatmap.png', dpi=150, bbox_inches='
    tight')
14 plt.show()

```

17.11 Cell 10 — Feature Engineering (Variance Filtering)

Listing 13: Cell 10: Zero-Variance Feature Removal

```

1 selector = VarianceThreshold(threshold=0.0)
2 selector.fit(X_train)
3
4 X_train_sel = selector.transform(X_train)
5 X_test_sel = selector.transform(X_test)
6 selected_features = X.columns[selector.get_support()].tolist()
7
8 print(f'Features before: {X_train.shape[1]} | After variance
9       filtering: {X_train_sel.shape[1]}')
10 print(f'Age feature retained: {"age" in selected_features}')

```

17.12 Cell 11 — Decision Tree Training and Evaluation

Listing 14: Cell 11: Decision Tree Classifier

```

1 # --- Decision Tree ---

```

```

2 dt_model = DecisionTreeClassifier(criterion='gini',
    random_state=42)
3 dt_model.fit(X_train_sel, y_train)
4
5 dt_pred = dt_model.predict(X_test_sel)
6 dt_acc = accuracy_score(y_test, dt_pred)
7
8 print(f'Decision Tree Accuracy: {dt_acc * 100:.2f}%')
9 print(classification_report(y_test, dt_pred, target_names=le.
    classes_))

```

17.13 Cell 12 — Random Forest Training and Evaluation

Listing 15: Cell 12: Random Forest Classifier (200 Trees)

```

1 # --- Random Forest (200 trees) ---
2 rf_model = RandomForestClassifier(n_estimators=200,
    random_state=42, n_jobs=-1)
3 rf_model.fit(X_train_sel, y_train)
4
5 rf_pred = rf_model.predict(X_test_sel)
6 rf_acc = accuracy_score(y_test, rf_pred)
7
8 print(f'Random Forest Accuracy: {rf_acc * 100:.2f}%')
9 print(classification_report(y_test, rf_pred, target_names=le.
    classes_))

```

17.14 Cell 13 — XGBoost and Stacking Ensemble Training

Listing 16: Cell 13: XGBoost Classifier and Stacking Ensemble

```

1 # --- XGBoost ---
2 xgb_model = XGBClassifier(
3     n_estimators=100, learning_rate=0.1, max_depth=6,
4     use_label_encoder=False, eval_metric='mlogloss',
5     random_state=42, verbosity=0
6 )
7 xgb_model.fit(X_train_sel, y_train)
8
9 xgb_pred = xgb_model.predict(X_test_sel)
10 xgb_acc = accuracy_score(y_test, xgb_pred)
11
12 print(f'XGBoost Accuracy: {xgb_acc * 100:.2f}%')
13 print(classification_report(y_test, xgb_pred, target_names=le.
    classes_))
14
15 # Stacking Ensemble (DT + RF + XGB -> LR meta-learner with
    passthrough)
16 # passthrough=True: meta-learner receives both the base model
    probability outputs

```

```

17 # AND the original feature set, letting it correct residual
    errors using raw signals.
18 print('\n--- Building Stacking Ensemble ---')
19
20 base_estimators = [
21     ('dt', DecisionTreeClassifier(criterion='gini',
22                                   random_state=42)),
22     ('rf', RandomForestClassifier(n_estimators=200,
23                                   random_state=42, n_jobs=-1)),
23     ('xgb', XGBClassifier(n_estimators=100, learning_rate=0.1,
24                             max_depth=6,
25                             use_label_encoder=False, eval_metric=
26                                 'mlogloss',
27                                 random_state=42, verbosity=0)),
28 ]
29
30 stack_model = StackingClassifier(
31     estimators=base_estimators,
32     final_estimator=LogisticRegression(max_iter=3000, C=0.1,
33                                         random_state=42, solver='saga'),
34     cv=StratifiedKFold(n_splits=5, shuffle=True, random_state
35                         =42),
36     stack_method='predict_proba',
37     passthrough=True,
38     n_jobs=-1
39 )
40 stack_model.fit(X_train_sel, y_train)
41
42 stack_pred = stack_model.predict(X_test_sel)
43 stack_acc = accuracy_score(y_test, stack_pred)
44
45 print(f'Stacking Ensemble Accuracy: {stack_acc * 100:.2f}%')
46 print(classification_report(y_test, stack_pred, target_names=le
47                             .classes_))

```

17.15 Cell 14 — Confusion Matrix Helper Function

Listing 17: Cell 14: Normalised Confusion Matrix Helper Function

```

1 def plot_confusion_matrix(y_true, y_pred, labels, model_name,
    filename):
2     cm = confusion_matrix(y_true, y_pred)
3     cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
4
5     fig, ax = plt.subplots(figsize=(16, 12))
6     sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues',
7                 xticklabels=labels, yticklabels=labels,
8                 linewidths=0.3, annot_kws={'size': 6})
9     ax.set_title(f'Confusion Matrix - {model_name} (Normalised)
    ', fontsize=13, fontweight='bold')

```

```

10 ax.set_xlabel('Predicted Label', fontsize=11)
11 ax.set_ylabel('True Label', fontsize=11)
12 plt.xticks(rotation=90, fontsize=7)
13 plt.yticks(rotation=0, fontsize=7)
14 plt.tight_layout()
15 plt.savefig(filename, dpi=150, bbox_inches='tight')
16 plt.show()

```

17.16 Cell 15 — Decision Tree Confusion Matrix

Listing 18: Cell 15: Decision Tree Confusion Matrix

```

1 plot_confusion_matrix(y_test, dt_pred, le.classes_, 'Decision
  Tree', 'cm_decision_tree.png')

```

17.17 Cell 16 — Random Forest Confusion Matrix

Listing 19: Cell 16: Random Forest Confusion Matrix

```

1 plot_confusion_matrix(y_test, rf_pred, le.classes_, 'Random
  Forest', 'cm_random_forest.png')

```

17.18 Cell 17 — XGBoost and Stacking Ensemble Confusion Matrices

Listing 20: Cell 17: XGBoost and Stacking Ensemble Confusion Matrices

```

1 plot_confusion_matrix(y_test, xgb_pred, le.classes_, 'XGBoost
  ', 'cm_xgboost.png')
2 plot_confusion_matrix(y_test, stack_pred, le.classes_, '
  Stacking Ensemble', 'cm_stacking.png')

```

17.19 Cell 18 — Model Accuracy Comparison Table

Listing 21: Cell 18: Model Accuracy Comparison Table

```

1 results_df = pd.DataFrame({
2     'Model': ['Decision Tree', 'Random Forest (200)', 'XGBoost',
3     'Stacking Ensemble'],
4     'Accuracy (%)': [
5         round(dt_acc * 100, 2),
6         round(rf_acc * 100, 2),
7         round(xgb_acc * 100, 2),
8         round(stack_acc * 100, 2),
9     ]
10 }).sort_values('Accuracy (%)', ascending=False).reset_index(
11     drop=True)
12 results_df.index += 1

```

```
12 print(results_df.to_string())
```

17.20 Cell 19 — Model Accuracy Bar Chart

Listing 22: Cell 19: Model Accuracy Comparison Bar Chart

```
1 colors = ['#4C72B0', '#55A868', '#C44E52', '#8172B2']
2
3 plt.figure(figsize=(9, 4))
4 bars = plt.bar(results_df['Model'], results_df['Accuracy (%)'],
5               color=colors[:len(results_df)], edgecolor='black',
6               width=0.5)
7
8 for bar in bars:
9     h = bar.get_height()
10    plt.text(bar.get_x() + bar.get_width() / 2, h - 2,
11            f'{h:.2f}%', ha='center', va='top', fontsize=11,
12            fontweight='bold', color='white')
13
14 plt.title('Model Accuracy Comparison (Age + Symptom Features,
15         Stacking Ensemble)',
16         fontsize=12, fontweight='bold')
17
18 plt.xlabel('Model')
19 plt.ylabel('Accuracy (%)')
20 plt.ylim(0, 110)
21 plt.tight_layout()
22 plt.savefig('model_comparison.png', dpi=150, bbox_inches='tight')
23 plt.show()
```

17.21 Cell 20 — Feature Importance Plot

Listing 23: Cell 20: Top 20 Feature Importances from Random Forest (incl. Age)

```
1 # Feature importance from the Random Forest base model (top 20)
2 importances = rf_model.feature_importances_
3 feat_names = np.array(selected_features)
4 indices = np.argsort(importances)[::-1][:20]
5
6 plt.figure(figsize=(12, 4))
7 sns.barplot(x=feat_names[indices], y=importances[indices],
8           palette='magma')
9 plt.title('Top 20 Feature Importances - Random Forest (incl.
10         Age)', fontsize=13, fontweight='bold')
11 plt.xlabel('Feature')
12 plt.ylabel('Importance Score')
13 plt.xticks(rotation=45, ha='right', fontsize=9)
14 plt.tight_layout()
15 plt.savefig('feature_importance.png', dpi=150, bbox_inches='tight')
```

```
14 plt.show()
```

17.22 Cell 21 — Final Results Summary

Listing 24: Cell 21: Final Results Summary

```
1 print('=' * 60)
2 print('      DISEASE PREDICTION SYSTEM - FINAL RESULTS')
3 print('      (Age + Symptoms | Stacking Ensemble)')
4 print('=' * 60)
5 print(f'  Decision Tree          Accuracy : {dt_acc      * 100:.2f}%')
6 print(f'  Random Forest (200)     Accuracy : {rf_acc      * 100:.2f}%')
7 print(f'  XGBoost                    Accuracy : {xgb_acc     * 100:.2f}%')
8 print(f'  Stacking Ensemble         Accuracy : {stack_acc * 100:.2f}% <-- BEST')
9 print('=' * 60)
10 best = results_df.iloc[0]
11 print(f'  Best Model : {best["Model"]} ({best["Accuracy (%)"]:.2f}%)')
12 print('=' * 60)
```